

INTERVIEW QUESTIONS TO LAND AN AI ENGINEER JOB

Transformer Architecture Interview Prep

If you work with LLMs, these fundamentals should feel natural.

Q01

Why are masked tokens used in encoder-based training objectives?

A

INTERVIEW-READY ANSWER

Masking creates a self-supervised training signal that forces the model to use bidirectional context to understand each token, which is exactly the capability needed for downstream understanding tasks.

POINT	WHAT IT MEANS
Creating a supervised signal without labels	Training a supervised model requires labeled data, which is expensive. Masking allows you to create effectively unlimited training examples from raw text with no human annotation. Mask "bank" in "The river bank was steep" and the label is "bank". You can do this for any text corpus.
Forcing deep contextual understanding	To predict the masked token correctly, the model must use all available surrounding context. For the sentence "She played the [MASK] at the concert last night", the model needs to understand that the context suggests a musical instrument, and concert context makes "violin", "piano", or "guitar" likely.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q01 Why are masked tokens used in encoder-based training objectives?**A** INTERVIEW-READY ANSWER · CONTINUED

⚡ This requires genuine semantic understanding of the sentence, not just local n-gram statistics.

POINT

WHAT IT MEANS

What the model learns

Through billions of these predictions during training, the model learns rich contextual representations where each token's meaning is shaped by its full context. This is exactly what makes BERT representations useful for downstream tasks: they encode contextual meaning, not just surface statistics.

The 80-10-10 trick

Original BERT uses an interesting modification: 80% of selected tokens are replaced with [MASK], 10% are replaced with a random token, 10% are left unchanged. The random and unchanged tokens prevent the model from learning to only update representations when it sees [MASK].

It must maintain good representations for all tokens since any could be the prediction target.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q02

Why is the attention score scaled by the square root of the key dimension?

A

INTERVIEW-READY ANSWER

This is one of those details that seems minor but has a real and important effect on training.

WHAT HAPPENS WITHOUT SCALING

The attention score for token i attending to token j is the dot product of query vector i and key vector j .

If the key dimension is d_k , and the vectors have values roughly normally distributed with mean 0 and variance 1, then the dot product has variance d_k (it is a sum of d_k multiplied random variables).

When d_k is large, say 64 or 128, the raw dot products can be quite large in magnitude, like 30 or 50. Softmax of values like $(-50, -30, 50, 30)$ is essentially $(0, 0, 1, 0)$. The distribution collapses to a one-hot vector.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q02

Why is the attention score scaled by the square root of the key dimension?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Why that is a problem

When softmax produces values very close to 0 or 1, its gradient is nearly zero. The model learns very slowly or stops learning. You want softmax to produce a range of intermediate values so gradients flow through all attention connections, not just the argmax.

The fix

Divide by square root of d_k . This normalizes the variance of the dot products back to approximately 1 regardless of dimension. Softmax operates in a range where it produces meaningful distributions and gradients are healthy.

Concretely: if d_k equals 64, divide scores by 8. If d_k equals 128, divide by roughly 11.3. As models get wider with larger head dimensions, the scaling factor grows accordingly.

Q03

Why are transformer layers stacked repeatedly, and how does depth influence model capability?

A

INTERVIEW-READY ANSWER

The stacking is not just about having more parameters. Each layer plays a qualitatively different role in the hierarchy of representations.

WHAT EACH LAYER LEARNS

Early layers tend to learn syntactic patterns: is this a noun or a verb? What are the grammatical dependencies between these words? Middle layers start learning more semantic patterns: what is the meaning of this phrase in context? Later layers capture task-relevant, abstract representations: what is the sentiment of this sentence?



What is the answer to this question?

We know this from attention visualization and probing experiments. If you train a simple classifier on the representations at each layer to predict something like POS tags, early layers are best for POS tagging. Later layers are best for semantic roles. The hierarchy is real and meaningful.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q03

Why are transformer layers stacked repeatedly, and how does depth influence model capability?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Why stacking works

Each layer's output becomes the next layer's input. Layer 2 does not operate on raw token embeddings. It operates on the enriched representations that layer 1 produced. This compositionality is what makes deep networks powerful. Complex patterns are built from simpler patterns in a hierarchy.

Depth vs width trade-off

You could make a single layer much wider (more neurons in the FFN, larger attention dimension). But empirically, depth beats width for the same parameter budget. A deep, moderately wide network generalizes better than a shallow, very wide network. The hierarchical processing that depth enables appears to match the hierarchical structure of language itself.

Practically, modern LLMs typically have 32 to 96 layers. GPT-3 has 96 layers. Each layer contributes meaningfully to the final representation.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q04

Why do decoder architectures use masked self-attention while encoders do not?

A

INTERVIEW-READY ANSWER

The masking is not a design preference. It is a strict requirement for autoregressive generation to work correctly.

WHY MASKING IS NECESSARY FOR GENERATION

When a decoder generates text, it produces one token at a time. At position 3, it generates token 3. At this point, tokens 4, 5, 6 do not exist yet. If the model attended to future positions during generation, it would be using information that is unavailable in practice.

More critically, if during training we let the model attend to future tokens, it would trivially solve the prediction task by just copying from ahead. The model would learn nothing useful because it can always see the answer.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q04

Why do decoder architectures use masked self-attention while encoders do not?

A

INTERVIEW-READY ANSWER · CONTINUED

HOW MASKING IS IMPLEMENTED

During training, all tokens are present simultaneously (for efficiency). The mask is applied as a large matrix that sets all upper-triangle attention scores to negative infinity before the softmax. This makes the attention weights for future positions exactly zero.



The model is forced to predict each position using only past positions, even though the entire sequence is technically present in memory.

WHY ENCODERS DO NOT NEED MASKING

Encoders are used for understanding, not generation. You give the model a complete input and ask it to produce a representation. There is no generation step where future tokens would be unavailable. Seeing the full bidirectional context is actually beneficial and produces better representations.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q05

Why do encoder-based models often require pooling mechanisms for downstream tasks?

A

INTERVIEW-READY ANSWER

Encoder models produce one representation vector per input token. Most downstream tasks need a single vector representing the entire input. Pooling is the bridge.

THE MISMATCH PROBLEM

Say you want to classify a 50-token sentence as positive or negative sentiment. BERT gives you 50 vectors, each representing a token's contextual meaning. But a classifier needs one vector as input. How do you go from 50 to 1?



That is what pooling solves. It aggregates across the sequence dimension to produce a fixed-size representation.

Q05

Why do encoder-based models often require pooling mechanisms for downstream tasks?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Why this design exists	BERT-style encoders were designed to produce rich per-token representations because they were trained on token-level tasks (MLM predicts individual masked tokens). The model was not explicitly trained to produce sequence-level representations. Pooling is the adapter that converts token-level representations to sequence-level representations for downstream classification, similarity, and retrieval tasks.
Why it matters for embedding models	When you use a model like sentence-BERT to produce a semantic embedding for a document, you are applying pooling over the encoder's token representations. The choice of pooling method directly affects the quality of the resulting embedding, which in turn affects retrieval quality in a vector search system.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q06

What tokenization strategies help mitigate out-of-vocabulary issues in language models?

A

INTERVIEW-READY ANSWER

Modern LLMs avoid out-of-vocabulary problems primarily through subword tokenization.

COMMON APPROACHES



Byte Pair Encoding (BPE)



Breaks rare words into smaller subword units.

EXAMPLE



suboptimality → sub + optim + ality



Byte-level BPE (used in GPT-2/3)



Works directly on byte sequences (0-255).

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q06

What tokenization strategies help mitigate out-of-vocabulary issues in language models?

A

INTERVIEW-READY ANSWER · CONTINUED

ADVANTAGES



any text can be represented



no unknown tokens



SentencePiece (used in T5, LLaMA)



Operates on raw text and learns subword units automatically.

BENEFITS



language-agnostic



handles multilingual text well

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q06

What tokenization strategies help mitigate out-of-vocabulary issues in language models?

A

INTERVIEW-READY ANSWER · CONTINUED

TYPICAL VOCABULARY SIZES IN LLMS



Model family



Vocabulary



BERT



~30k



GPT-2/3



~50k



LLaMA

~32K

These tokenization approaches ensure that even unseen words can be decomposed into known subwords, eliminating true OOV failures.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q07

Why has RoPE become common in modern LLM architectures?

A

INTERVIEW-READY ANSWER

Several practical advantages have made RoPE the dominant choice in post-2022 architectures.

POINT	WHAT IT MEANS
Better length generalization	As I mentioned, RoPE's relative position formulation allows the model to handle sequences longer than those seen during training more gracefully than additive sinusoidal encoding. This is crucial because:
Training on very long sequences is expensive	Production use cases often need longer context than what was practical to train on



Users want to be able to process long documents without the model falling apart

COMPATIBILITY WITH KV CACHING

In inference, key and value vectors are cached as context grows (this is the KV cache that makes autoregressive generation efficient). RoPE applies the rotation to keys and queries in a way that is compatible with incremental KV cache updates.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q07

Why has RoPE become common in modern LLM architectures?

A

INTERVIEW-READY ANSWER · CONTINUED



The positional rotation can be applied at the time of caching and correctly interacts with later queries.

POINT

WHAT IT MEANS

Scalable context extension

Techniques like YaRN, LongRoPE, and various other RoPE scaling methods allow models to extend their context window after training with relatively small amounts of fine-tuning. A LLaMA-2 model trained on 4k context has been extended to 128k context using RoPE interpolation techniques. This flexibility is very valuable practically.

It just works well empirically

Performance on benchmarks that test long-context understanding is consistently better with RoPE than with additive positional encoding, which has driven rapid adoption.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q08

What challenges arise when training MoE transformer architectures?

A

INTERVIEW-READY ANSWER

MoE training introduces complications that do not exist in dense model training, and several of them are tricky to get right.

EXPERT COLLAPSE

Without load balancing, gradient descent finds a shortcut: always route to the 1-2 experts that perform best and ignore the rest. The underutilized experts get no gradient signal and never improve. The model is a MoE in structure but a tiny dense model in practice.

The auxiliary load balancing loss is the standard solution but adds training complexity. Too much auxiliary loss and routing optimization dominates. Too little and collapse happens anyway.

TRAINING INSTABILITY

MoE models are notoriously more unstable during training than equivalent dense models. Loss spikes, NaN values, and divergence are more common. This is partly because the routing decisions create discrete jumps in the computational graph that do not behave smoothly.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q08

What challenges arise when training MoE transformer architectures?

A

INTERVIEW-READY ANSWER · CONTINUED

Careful tuning of learning rates, gradient clipping, expert capacity, and router initialization is required. Many practitioners use lower learning rates and more aggressive gradient clipping for MoE than for dense models of similar capability.

POINT

WHAT IT MEANS

Token dropping

When an expert receives more tokens than its capacity buffer, excess tokens are dropped (processed with only the residual connection, no expert computation). During training, high drop rates mean some tokens receive no expert processing, which degrades gradient quality and final model quality. Managing expert capacity requires careful calibration.

Communication overhead in distributed training

The all-to-all communication required to route tokens to experts across devices adds latency to each training step. In large clusters, this communication can become a significant bottleneck. Efficient implementation with proper overlapping of computation and communication is required to achieve good hardware utilization.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q09

How do transformers learn contextual token relationships during training?

A

INTERVIEW-READY ANSWER

This is really asking about what gradient descent discovers in the attention and FFN weights that enables contextual understanding.

ATTENTION WEIGHT LEARNING

The query, key, and value weight matrices (W_q , W_k , W_v) are learned parameters. During training, the model gets feedback (gradient) about which attention patterns led to correct predictions.

If attending to the subject token strongly when encoding a verb consistently reduces the language modeling loss (because subject-verb agreement is important), the model adjusts W_q and W_k so that verb queries produce high dot products with subject keys. This is not programmed; it emerges from millions of gradient updates over the training data.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q09

How do transformers learn contextual token relationships during training?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Multi-head specialization	Different attention heads converge to different specializations because they start with different random initializations and follow different gradient paths. One head may find it useful to track grammatical subjects. Another finds tracking semantic similarity useful. The gradient descent independently optimizes each head for whatever attention pattern reduces loss most.
FFN weight learning	FFN layers learn to store and retrieve knowledge. For a language modeling objective, predicting "The Eiffel Tower is located in [MASK]" correctly requires knowing that Paris follows Eiffel Tower. The FFN weights store these associations as patterns in the weight matrices.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q09

How do transformers learn contextual token relationships during training?

A

INTERVIEW-READY ANSWER · CONTINUED



Research has shown that specific facts can be localized to specific FFN neurons in transformer models.

LAYER-BY-LAYER ABSTRACTION

Earlier layers learn shallower patterns (word identity, local context). Later layers learn more abstract patterns (semantic roles, factual associations). This hierarchy emerges naturally from the depth of the network and the training objective.



7. Mixture of Experts Transformers

Q10

How do residual connections and layer normalization stabilize transformer training, and how do Pre-LN and Post-LN architectures differ?

A

INTERVIEW-READY ANSWER

Residual connections provide a direct path for gradients during backpropagation, preventing gradient vanishing in deep networks.

INSTEAD OF COMPUTING

🛡️ $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$

MODERN TRANSFORMERS TYPICALLY USE PRE-LAYERNORM

🗄️ $\text{output} = x + \text{Sublayer}(\text{LayerNorm}(x))$

WHY PRE-LN BECAME STANDARD



Better gradient stability in very deep models



Enables training transformers with 100+ layers

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q10

How do residual connections and layer normalization stabilize transformer training, and how do Pre-LN and Post-LN architectures differ?

A

INTERVIEW-READY ANSWER · CONTINUED

REDUCES TRAINING DIVERGENCE

Post-LN (original Transformer paper) works but becomes unstable in very deep architectures.

RESIDUAL CONNECTIONS ENSURE

$$\text{OUTPUT} = X + F(X)$$

This gives a direct gradient path of 1, preventing gradient shrinkage across many layers.

Layer normalization stabilizes training by normalizing feature activations across the hidden dimension for each token.

Q11

How does expert routing affect model efficiency and training stability?

A

INTERVIEW-READY ANSWER

Expert routing is the mechanism that determines both the efficiency gains (good routing means efficient computation) and stability risks (bad routing means training problems).

EFFICIENCY DEPENDS ON ROUTING QUALITY

If the router consistently sends related tokens to the same experts, those experts develop specialized knowledge. A routing pattern where medical tokens go to expert 3 and legal tokens go to expert 7 allows those experts to develop domain specialization.



This specialization improves performance and efficiency: each expert becomes very good at its domain with fewer parameters than a generalist expert of the same size.



If routing is random or uniform (every expert sees everything equally), no specialization happens and you lose much of the MoE benefit.

Q11

How does expert routing affect model efficiency and training stability?

A

INTERVIEW-READY ANSWER · CONTINUED

THE DISCRETE ROUTING PROBLEM

Hard routing (assign each token to exactly K experts) creates a non-differentiable operation. The router makes a discrete selection that cannot be differentiated with respect to the router's weights in the standard sense. This is why the auxiliary load balancing loss is applied in the continuous routing probability space rather than the discrete selection.

Some approaches use soft routing (weighted combination of all experts for each token) during training and hard routing during inference. This improves gradient flow but increases training compute.

STABILITY THROUGH CAPACITY FACTORS

The expert capacity factor (how many tokens each expert can process per batch relative to an equal distribution) is a key hyperparameter. Too tight (capacity factor 1.0) and any imbalance causes token dropping and instability. Too loose (capacity factor 2.0) wastes memory and compute.



Most practitioners use capacity factors between 1.0 and 2.0 with monitoring of token drop rates throughout training.



8. Practical Training Challenges

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q12

What problem does the Mixture of Experts architecture attempt to solve in large transformer models?

A

INTERVIEW-READY ANSWER

The core problem is that dense transformer models use all their parameters for every input token, which becomes inefficient and expensive as you scale to hundreds of billions of parameters.

THE SCALING CHALLENGE

A 100-billion-parameter dense model processes every input token through all 100 billion parameters. The computation cost scales directly with parameter count. Doubling parameters doubles computation, doubles memory, doubles cost.

But do you really need all 100 billion parameters to process every token? A simple common word like "the" probably does not need the same computation as a complex technical term. A question about cooking does not need the same knowledge as a question about quantum physics.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q12

What problem does the Mixture of Experts architecture attempt to solve in large transformer models?

A

INTERVIEW-READY ANSWER · CONTINUED

WHAT MOE DOES DIFFERENTLY

MoE replaces the single FFN in each transformer block with multiple "expert" FFNs (typically 8, 16, or 64 experts). A lightweight routing network selects only 1 or 2 experts to process each token. The other experts are completely skipped.

This means a model with 8 experts, each the size of a standard FFN, has 8x the total parameters but only activates 1/8 of them per token. Total parameter count is large (enabling knowledge storage) but computation per token stays approximately constant.

THE PRACTICAL PAYOFF

Mixtral 8x7B has 46 billion total parameters but only activates about 13 billion per token. It achieves performance competitive with much larger dense models while matching the inference speed of a 13B dense model. You get the knowledge capacity of a large model with the computation cost of a small model.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q13

Why do transformers require positional encoding to process sequences?

A

INTERVIEW-READY ANSWER

This is a subtle but important point. Without positional encoding, a transformer is completely order-blind.

THE PERMUTATION INVARIANCE PROBLEM

Self-attention computes relationships based entirely on the content of token embeddings. It does not look at positions anywhere in its computation. The attention score between "cat" and "sat" is the same whether "cat" comes before "sat" or after "sat".

If you shuffled all the tokens in a sentence and fed it to a transformer without positional encoding, you would get the exact same output representations (just shuffled the same way). The model cannot distinguish "dog bites man" from "man bites dog".

THE FIX: INJECT POSITION INFORMATION

Positional encoding adds a position-specific vector to each token embedding before the first transformer layer. Token 1 gets embedding + position_vector_1. Token 2 gets embedding + position_vector_2. Now the input vectors encode both meaning and position.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q13

Why do transformers require positional encoding to process sequences?

A

INTERVIEW-READY ANSWER · CONTINUED

Because the attention mechanism operates on these enriched vectors, the attention scores now implicitly incorporate positional information. "cat" at position 1 has a slightly different vector than "cat" at position 10, so their interaction patterns with surrounding tokens will differ.

TWO MAIN APPROACHES

Absolute positional encoding adds fixed (sinusoidal) or learned vectors to each position. Simple and effective for sequences up to the training length.

Relative positional encoding encodes the distance between tokens rather than their absolute positions. More generalizable to sequences longer than seen during training. RoPE is the most popular modern variant.

Q14

Why can the CLS token represent the entire sequence in BERT-style models?

A

INTERVIEW-READY ANSWER



The [CLS] token is placed at the beginning of the sequence.

DURING SELF-ATTENTION



[CLS] attends to all tokens

ALL TOKENS CAN ATTEND TO [CLS]

Through multiple transformer layers, [CLS] accumulates information from the entire sequence.



The key reason it becomes a sequence representation is training supervision.

DURING



Next Sentence Prediction (in original BERT pretraining)

Q14

Why can the CLS token represent the entire sequence in BERT-style models?

A

INTERVIEW-READY ANSWER · CONTINUED



downstream classification tasks



the loss is applied to the CLS representation.



Because gradients flow through this representation, the model learns to encode sequence-level information inside the CLS token.

HOWEVER

CLS works best after task-specific fine-tuning



for sentence embeddings, mean pooling often performs better

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15

Can you explain the full transformer architecture from input embeddings to output predictions?

A

INTERVIEW-READY ANSWER

Sure, let me walk through it step by step the way it actually works during a forward pass.

POINT	WHAT IT MEANS
Step 1: Tokenization and embedding	The raw text first gets tokenized into sub-word tokens. Each token is mapped to an integer ID, and that ID is looked up in an embedding matrix to get a dense vector, typically 512 or 768 dimensions depending on the model. So the input becomes a matrix of shape (sequence length x embedding dimension).
Step 2: Positional encoding	Since the transformer has no built-in sense of order, we add positional information directly to the token embeddings. Each position gets a unique positional vector added to it, so the model can distinguish "cat sat" from "sat cat".

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15

Can you explain the full transformer architecture from input embeddings to output predictions?

A

INTERVIEW-READY ANSWER · CONTINUED



The result is still a (sequence length x embedding dimension) matrix, but now each vector encodes both meaning and position.

STEP 3: TRANSFORMER BLOCKS

This is where the real work happens. The enriched embeddings pass through N identical transformer blocks stacked on top of each other. Each block contains:



A self-attention sublayer where every token looks at every other token and updates its representation based on what is relevant



A feed-forward sublayer that applies a small MLP independently to each token position



Residual connections that add the input of each sublayer back to its output

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15

Can you explain the full transformer architecture from input embeddings to output predictions?

A INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Layer normalization that stabilizes the scale of activations

After each block, the representations get richer and more contextual. Token representations in deeper layers encode more abstract, task-relevant information.

Step 4: Final output

For a language model, the final block's output (sequence length x embedding dimension) is projected through a linear layer to vocabulary size, giving a (sequence length x vocab size) matrix of logits. Softmax converts these to probabilities over the vocabulary.



The token with the highest probability at the last position is the next predicted token.



For classification tasks like BERT, just the CLS token representation from the final layer is passed to a small classifier head.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q16

Why does the transformer architecture rely entirely on attention instead of recurrence or convolution?

A

INTERVIEW-READY ANSWER

This is a really good question because it gets at the core design philosophy of transformers.

THE PROBLEM WITH RECURRENCE (RNNs)

RNNs process tokens one at a time. To get from token 1 to token 100, information has to travel through 99 sequential steps. Two problems happen here. First, the gradient has to backpropagate through all those steps, which causes vanishing gradients. Second, you cannot parallelize training because step N depends on step N-1.



For long sequences this is both slow and lossy.

Q16

Why does the transformer architecture rely entirely on attention instead of recurrence or convolution?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
The problem with convolution (CNNs)	CNNs apply fixed-size kernels that look at local neighborhoods. A kernel of size 3 sees 3 adjacent tokens. To model dependencies between tokens that are far apart, you need many stacked layers, and even then the receptive field grows slowly. They are great for local patterns but awkward for long-range relationships.
Why attention solves both problems	Attention computes a direct relationship between every pair of tokens in a single operation. Token 1 and token 100 have the same connection cost as tokens 1 and 2. There is no distance penalty. And the entire attention matrix is computed in parallel as a single matrix multiplication, so training is dramatically faster.

The practical payoff is huge. A model trained on GPUs can process the entire sequence simultaneously rather than sequentially. And a word at position 1 can directly influence the representation of a word at position 500 without degradation. That is simply not possible with RNNs at scale.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q17

How does causal language modeling differ from masked language modeling?

A

INTERVIEW-READY ANSWER

These two objectives represent fundamentally different approaches to self-supervised training on text, and they produce models with different strengths.

CAUSAL LANGUAGE MODELING (GPT-STYLE)

The model predicts each token from all previous tokens. Given "The cat sat on the", predict "mat". Given "The cat sat on the mat", predict the next token. The training loss is the average cross-entropy over every position in the sequence.



The attention mask is causal: each position only attends to prior positions. This is necessary to prevent the model from looking ahead and trivially solving the task.



This objective trains the model to be a good text generator. It learns the full joint distribution of text by learning to continue any prefix.

Q17

How does causal language modeling differ from masked language modeling?

A

INTERVIEW-READY ANSWER · CONTINUED

MASKED LANGUAGE MODELING (BERT-STYLE)

Randomly mask 15% of tokens, replacing them with a special [MASK] token. The model predicts the masked tokens from the surrounding unmasked context. The training loss is computed only on the masked positions.

The attention mask is fully bidirectional: each position can attend to all other unmasked positions. This allows the model to use both left and right context when predicting a masked token.

This objective trains the model to understand context deeply.



Predicting a masked word requires understanding what word makes sense given everything around it.

KEY PRACTICAL DIFFERENCES

CLM trains on 100% of tokens in every sequence (every position is a prediction target). MLM trains on only about 15% (only the masked positions). CLM is more sample-efficient from this perspective.

CLM produces autoregressive generative models. MLM produces bidirectional understanding models. They optimize for different end goals.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18

What architectural role does the feed-forward network play inside a transformer block?

A

INTERVIEW-READY ANSWER

A lot of people focus entirely on attention and treat the feed-forward network as an afterthought. That is a mistake. The FFN plays a distinct and important role.

WHAT THE FFN ACTUALLY DOES

After self-attention, each token has a new representation that incorporates information from other tokens in the sequence. The FFN then processes each of those representations independently. It applies the same two-layer MLP to every position:



Linear layer: expands from model dimension (say 768) to 4x that (3072)



Non-linear activation (GELU or ReLU)



Linear layer: projects back down to model dimension (768)

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18

What architectural role does the feed-forward network play inside a transformer block?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Why the 4x expansion matters	The expansion creates a higher-dimensional space where the model can perform more complex non-linear transformations before projecting back. Think of it like working in a larger workspace, doing computation there, then summarizing back to the original dimension.
Attention vs FFN: what each does	Self-attention is fundamentally about routing: it figures out which other tokens are relevant and mixes their information together. It answers the question "what context should I incorporate?"

The FFN is about computation on the result: given this contextualized representation, what transformation should I apply? Research has shown that FFN layers store factual knowledge. When you see a model completing "The Eiffel Tower is located in..." the FFN layers are where the stored association "Eiffel Tower: Paris" gets activated.

This is why removing FFN layers significantly degrades factual recall even when attention layers are kept intact.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q19

What architectural differences exist between MoE transformers and dense transformers?

A

INTERVIEW-READY ANSWER

The differences are mostly contained to the FFN sublayer within each transformer block.

POINT	WHAT IT MEANS
Dense transformer FFN	Every transformer block has one FFN: a two-layer MLP with expand-activate-project. All tokens pass through this FFN in every forward pass. Parameters: 2 times (d_{model} times d_{ffn}).
MoE transformer FFN	The single FFN is replaced by N expert FFNs plus a router network. Each expert has the same structure as a standard FFN. The router adds a small linear layer (d_{model} times N_{experts}). Only top- K experts activate per token.



Total FFN parameters: N times 2 times (d_{model} times $d_{\text{ffn_expert}}$). But compute per token: K times 2 times (d_{model} times $d_{\text{ffn_expert}}$) where K is much less than N .

Q19

What architectural differences exist between MoE transformers and dense transformers?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Everything else stays the same	The attention mechanism, positional encoding, residual connections, layer normalization, and embedding layers are identical to dense transformers. Only the FFN sublayer changes.
Distributed training requires expert parallelism	In a dense model, you shard the model by layers (pipeline parallelism) or by matrices (tensor parallelism). In a MoE model, experts naturally distribute across devices. Different GPUs or nodes host different experts.

The routing step requires an all-to-all communication operation to send each token to its assigned expert, which may be on a different device.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q20

What practical limitations exist with sinusoidal positional encoding in long-context transformers?

A

INTERVIEW-READY ANSWER

Sinusoidal positional encoding provides deterministic positional patterns that extend to arbitrary sequence lengths.

HOWEVER, PRACTICAL LIMITATIONS INCLUDE

POINT	WHAT IT MEANS
Generalization beyond training length	Models trained on sequences up to length N often perform poorly on sequences much longer than N, even though sinusoidal encodings exist mathematically.
No explicit relative position modeling	Sinusoidal encoding represents absolute position, while many language patterns depend on relative distances between tokens.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q20

What practical limitations exist with sinusoidal positional encoding in long-context transformers?

A INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Limited
flexibility

The fixed encoding cannot adapt during training, unlike learned positional embeddings or rotary methods.

BECAUSE OF THESE LIMITATIONS, MODERN LLMS OFTEN USE



RoPE (Rotary Positional
Encoding)



ALiBi



learned positional
embeddings

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

This is only a selected preview.

Get the complete AI/GenAI Engineer Interview Kit for deeper preparation across the full curriculum.

 Full question bank

 Detailed answers

 Follow-up questions

 Real-world scenarios

↓ Check the caption to enroll

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT